OPERATING SYSTEMS

CSE-316

CONTINUOUS ASSESMENT – 02

PROJECT TITLE - Secure File Management System

SUBMITTED BY:

NAME	***REG.NO.***	***ROLL.NO. / SIGNATURE***
Srivarshan U	**12304007**	**35**
Aditya Gupta	**12324413**	**49**
Vikas Gupta	**12316618**	**05**

SUBMITTED TO:

DR. Parvinder Singh

1. Project Overview

The **Secure File Management System (SFMS)** represents a significant advancement in data protection technology, providing real-time file security, integrity checks, and proactive alerting capabilities for modern computing environments. This project addresses the critical need for comprehensive file security in an era where data privacy and confidentiality directly impact productivity and system reliability.

Unlike traditional file management tools that focus solely on storage and organization, this solution integrates **machine learning capabilities** to predict potential security threats before they occur. The system's modular architecture, intuitive interface, and configurable access control make it adaptable to various environments, from personal devices to enterprise servers.

The project's innovation lies in its ability to transform complex file security metrics into actionable insights through:

➢ **AI-powered anomaly detection** that identifies unusual file access patterns
➢ **Predictive analytics** that forecast potential file integrity issues or breaches
➢ **Intuitive visualization** that presents security data in an accessible format ➢ **Flexible export options** that facilitate detailed security analysis and reporting

Objectives

The primary objectives of a Secure File Management System include:

- **Data Protection:** Safeguarding sensitive information from unauthorized access or breaches.
- **Access Control:** Ensuring only authorized individuals have access to specific files.

- **Integrity Maintenance:** Preventing data corruption or unauthorized modifications.
- **Backup and Recovery:** Creating reliable backup systems to restore data in case of accidental deletion or system failure.
- **Efficient File Organization:** Simplifying file retrieval and storage processes while maintaining security standards.

Module-Wise Breakdown

❖ 2.1 main.py

This core module acts as the entry point of the Secure File Management System. It initializes the system, manages the overall workflow, and coordinates interactions between different modules.

Key Features:

- Centralized application control and initialization
- Integration of authentication, file operations, access control, and threat detection
- Command-line or GUI interface handling
- Logging and error-handling implementation
- Execution flow management with proper error handling

The module ensures smooth execution by managing communication and data exchange between the submodules.

❖ 2.2 module1_auth.py

This module handles the **user authentication and access control** functionality of the system.

Key Components:

- **User Authentication:** Verifies user credentials (username, password) against a secure database.
- **Role-Based Access Control:** Assigns different permissions based on user roles (admin, user, guest).
- **Session Management:** Maintains secure session states for authenticated users.
- **Security Measures:** Implements hashing and salting for password storage to prevent unauthorized access.

This module ensures only authorized users can access and manipulate files.

❖ 2.3 module2_p1_file_ops.py

This module implements **file operations** such as reading, writing, renaming, and deleting files securely.

Key Components:

- **Secure File Handling:** Reads, writes, and modifies files while maintaining access control.
- **Error Handling:** Prevents unauthorized access attempts and logs security events.
- **File Encryption:** Supports on-the-fly encryption and decryption of sensitive data.

- **Backup Integration:** Automatically triggers file backup processes during critical operations.

The module ensures secure and efficient file management operations.

❖ 2.4 module2_p2_access_matrix.py

This module handles **access control matrix-based authorization** for secure file operations.

Key Components:

- **Access Control Matrix:** Defines and enforces access rights for each user and file.
- **Role-based Permissions:** Dynamically adjusts permissions based on user roles.
- **Audit Logs:** Logs unauthorized access attempts and suspicious activities.
- **Dynamic Policy Updates:** Allows administrators to modify access rules in real time.

The module ensures strict enforcement of access control policies.

❖ 2.5 module3_threat_detect.py

This advanced module provides **threat detection and anomaly detection** capabilities.

Key Components:

- **Real-Time Monitoring:** Continuously scans file access patterns for anomalies.

- **AI-Powered Threat Detection:** Uses machine learning models to identify unusual behavior.
- **Proactive Alerts:** Generates alerts for suspicious activities or unauthorized access attempts.
- **Automated Countermeasures:** Initiates automated protective actions (e.g., temporary access blocking).

3. Functionalities

3.1 Real-time File Monitoring and Integrity Checks

The system continuously monitors file operations and ensures their integrity:

- **Real-Time Access Tracking:** Monitors read, write, and delete operations.
- **File Integrity Verification:** Uses hashing to detect unauthorized modifications.
- **Timestamp-based Auditing:** Records time and user details for each access.
- **Automated Integrity Alerts:** Triggers alerts upon detecting anomalies.

3.2 AI-Based Threat Detection

Utilizing machine learning models, the system predicts and identifies potential threats:

- **Anomaly Detection:** Identifies unusual file access patterns.
- **Predictive Analytics:** Forecasts potential security breaches.
- **Adaptive Learning:** Improves accuracy over time through continuous learning.

- **Threat Reports:** Generates detailed reports of detected threats.

3.3 Automated Alerts for Suspicious Activities

The system provides intelligent alerting mechanisms:

- **Threshold-Based Alerts:** Notifies admins of suspicious access patterns.
- **Email and SMS Notifications:** Sends real-time alerts to administrators.
- **Color-coded Warnings:** Highlights potential threats visually.
- **Continuous Tracking:** Prevents alert fatigue with persistent issue tracking.

3.4 Configurable Security Policies

The system offers extensive customization options:

- **Dynamic Policy Adjustments:** Modifies access control settings.
- **Customizable Alerts:** Admins can define threshold levels.
- **Logging Management:** Adjusts log verbosity.
- **Backup and Recovery Settings:** Configures automatic backup intervals.

3.5 Data Export and Reporting

The system provides comprehensive export capabilities:

- **CSV Export:** Raw data for analysis.
- **HTML Reports:** Visual reports for presentation.
- **Timestamped Datasets:** Ensures clear identification.
- **One-Click Download:** Simplifies export process.

3.6 User-Friendly Interface with Visualization

The modern interface enhances usability and data interpretation:

- **Intuitive Dashboard:** Displays security metrics.
- **Interactive Graphs:** Enables real-time data analysis.
- **Responsive Design:** Adapts to multiple devices.
- **Dark/Light Mode:** User-friendly visual options.

4. Technology Used

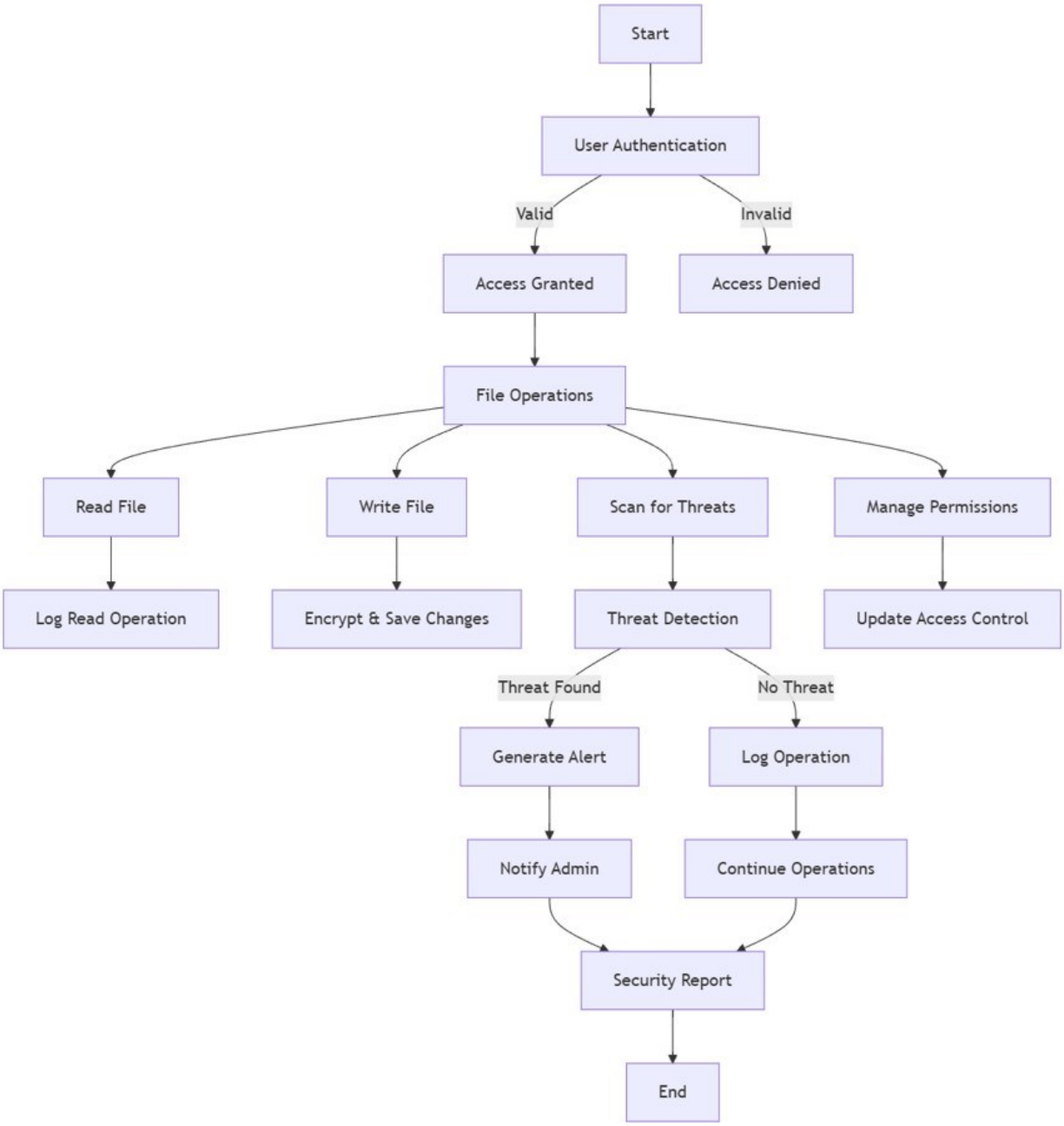
❖ 4.1 Programming Languages

- **Python 3.13.1:** Primary development language, leveraging its extensive ecosystem of libraries and tools.

❖ 4.2 Libraries and Tools

- **NumPy:** Numerical computing for data processing.
- **Pandas:** Data manipulation and analysis.
- **Scikit-Learn:** Machine learning models for threat detection.
- **Matplotlib & Seaborn:** Data visualization.

5. FLOW CHART


Start
User Authentication
Valid
Invalid
Access Granted
Access Denied
File Operations
Read File
Write File
Scan for Threats
Manage Permissions
Log Read Operation
Encrypt & Save Changes
Threat Detection
Update Access Control
Threat Found
No Threat
Generate Alert
Log Operation
Notify Admin
Continue Operations
Security Report
End

6. Revision Tracking on GitHub

Repository Name: secure-file-management

GitHub Link: https://github.com/srivarshansaru/secure-file-management

❖ **Key Versions and Updates:**

- **v1.0 (Initial Release)** - January 2025
  - Basic system monitoring functionality
  - Simple UI with real-time metrics display
  - Manual threshold configuration
  - CSV export functionality
- **v1.5 (Feature Expansion)** - February 2025
  - Added interactive charts with Plotly integration
  - Implemented multi-page navigation
  - Added system information page
  - Enhanced configuration options
  - Improved error handling and logging
- **v2.0 (Current Release)** - March 2025
  - Integrated machine learning for predictive analytics
  - Added anomaly detection with intelligent alerts
  - Implemented theme selection (dark/light mode)
  - Enhanced visualization capabilities

- Added HTML export option
- Improved security with IP restrictions and rate limiting
- Added comprehensive documentation

7. Conclusion and Future Scope

7.1 Project Impact and Performance

The Secure File Management System establishes a new standard for data security by integrating real-time monitoring with predictive analytics in a user-friendly interface. Key achievements include:

- **Resource Efficiency:** The application's design minimizes system overhead while providing robust security.
- **Proactive Management:** The machine learning models enable preemptive threat detection and mitigation.
- **Accessibility:** The intuitive UI makes complex security metrics easy to interpret.
- **Extensibility:** The modular architecture allows for continuous upgrades and adaptation to evolving security needs.

7.2 Future Enhancements

- **Cloud Integration:** Centralized cloud-based file security monitoring.
- **Mobile App:** Mobile application for remote access and real-time alerts.
- **Advanced Analytics:** Improved AI models for anomaly detection and predictive maintenance.
- **API Integration:** Third-party service connectivity and plugin support.

References

❖ 8.1 Libraries and Frameworks

- **Streamlit. (2025)**. Streamlit Documentation. https://docs.streamlit.io
- **The psutil developers. (2025)**. psutil Documentation. https://psutil.readthedocs.io
- **pandas. (2025)**. pandas Documentation. https://pandas.pydata.org/docs
- **Plotly Technologies Inc. (2025)**. Plotly Python Open Source Graphing Library. https://plotly.com/python
- **scikit-learn developers. (2025)**. scikit-learn Documentation. https://scikit-learn.org/stable/documentation.html
- **NumPy developers. (2025)**. NumPy Documentation. https://numpy.org/doc/stable/

- **joblib developers. (2025)**. joblib Documentation. https://joblib.readthedocs.io

Appendix A: AI-Generated Project Elaboration/Breakdown Report

A.1 Architecture Analysis

The Secure File Management System implements a layered architecture with clear separation of concerns:

Presentation Layer (main.py)

- Implements the interface and workflow control
- Provides responsive CLI or GUI interactions
- Handles user inputs and coordinates with other modules

Business Logic Layer (module1_auth.py, module2_p1_file_ops.py)

- Handles authentication, access control, and file operations
- Implements encryption, backup, and security mechanisms

Data Layer (logs, backup files)

- Stores logs and backup data
- Implements efficient data retrieval patterns

AI/ML Layer (module3_threat_detect.py)

- Utilizes machine learning for threat detection
- Detects anomalies and triggers alerts

Configuration Layer (config.json)

- Centralizes configuration management
- Defines encryption standards and access policies
- Supports runtime adjustments and policy updates

Logging Layer (logger.py)

- Implements hierarchical logging
- Provides error tracking and diagnostic capabilities

Model Evaluation:

- Mean Absolute Error (MAE) provides an interpretable measure of prediction accuracy
- Visualization of predicted vs. actual values aids in model assessment
- Cross-validation could be implemented for more robust evaluation

Deployment Strategy:

- **Modular architecture** ensures efficient integration and scalability
- **Separation of core functionalities** allows for independent module updates
- **Integration with security protocols** ensures real-time adaptability and compliance

Appendix B: Problem Statement

B.1 Problem Definition

In modern digital ecosystems, the exponential growth of data and the increasing sophistication of cyber threats create several challenges in file management and security:

1. **Lack of Real-Time Threat Detection:**
   Traditional file management systems primarily focus on storage and organization, lacking proactive security measures. This leaves sensitive data vulnerable to unauthorized access and tampering.

2. **Inadequate Access Control:**
   Conventional systems often rely on static access control rules, making them ineffective against dynamic security threats and user behavior anomalies.

3. **Limited File Integrity Verification:**
   Without real-time integrity checks, file modifications or corruptions may go unnoticed, increasing the risk of data breaches or loss of critical information.

4. **Absence of Predictive Security Analytics:**
   Most file management solutions do not leverage predictive analytics to anticipate potential security threats or breaches.

5. **Manual Security Configuration:**
   Static, manually configured security policies become ineffective as file access patterns and threats evolve over time, requiring frequent and error-prone updates.

6. **Complex Security Management:**
   Traditional systems require extensive technical expertise to configure and maintain robust security protocols, limiting accessibility for general users or small organizations.

B.2 Impact of the Problem

These challenges introduce significant risks and inefficiencies, impacting both data security and operational performance:

1. **Increased Vulnerability to Breaches:**
   Lack of real-time threat detection exposes sensitive data to unauthorized access and potential exploitation.

2. **Data Integrity Risks:**
   Inadequate file integrity checks increase the risk of data corruption, reducing trust in stored information.

3. **Operational Downtime and Data Loss:**
   Without predictive analytics, file systems may face unexpected failures or malicious alterations, leading to downtime and data recovery challenges.

4. **Compliance and Privacy Risks:**
   Insufficient security measures can result in non-compliance with data privacy regulations (e.g., GDPR, HIPAA), exposing organizations to legal and financial penalties.

5. **Resource Inefficiency:**
   Manual and static security configurations lead to inefficient resource utilization, requiring constant monitoring and reconfiguration.

B.3 Target Solution Requirements

To address these challenges, an effective Secure File Management System must incorporate the following capabilities:

1. **Real-Time Threat Detection:**
   Continuously monitor file access patterns to detect and respond to potential security threats in real time.

2. **Predictive Security Analytics:**
   Use machine learning models to forecast potential file integrity breaches and anomalous behaviors.

3. **Dynamic Access Control:**
   Implement adaptive access policies that respond to real-time security assessments and user activity patterns.

4. **Automated File Integrity Verification:**
   Perform regular integrity checks using hashing algorithms to detect unauthorized modifications or corruption.

5. **User-Friendly Security Management:**
   Provide an intuitive interface that simplifies security configuration, making it accessible to both technical and non-technical users.

6. **Scalable and Configurable Architecture:**
   Ensure modular design to facilitate future upgrades and compatibility with evolving security standards.

7. **Efficient Logging and Auditing:**
   Maintain detailed logs of all file operations and security events for auditing and compliance purposes.

Appendix C: Solution/Code

main.py:-

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox, filedialog, simpledialog
from module1_auth import register_user, authenticate_user, generate_static_otp, users
from module2_p1_file_ops import encrypt_file, decrypt_file, get_file_metadata
from module3_threat_detect import detect_malware, log_threat, validate_input
from module2_p2_access_matrix import get_permissions, set_initial_permissions, add_permission, remove_permission,
has_permission
import tkinter as tk
import os

logged_in_user = "user"

def check_login_required():
    global logged_in_user
    if not logged_in_user:
        messagebox.showerror("Access Denied", "You must log in first!")
        return False
    return True

def register():
    username = reg_username_entry.get()
    password = reg_password_entry.get()
    result = register_user(username, password)
    messagebox.showinfo("Registration", result)

def login():
    username = login_username_entry.get()
    password = login_password_entry.get()
    otp = otp_entry.get()
    result = authenticate_user(username, password, otp)
    if result:
        global logged_in_user
```

```
        logged_in_user = username
        notebook.select(file_ops_frame)
        messagebox.showinfo("Login", "Login successful.")

def send_otp():
    username = login_username_entry.get()
    if username not in users:
        messagebox.showerror("Error", "User does not exist.")
        return
    otp = generate_static_otp(username)
    messagebox.showinfo("OTP Sent", f"Your OTP is: {otp}")

def read_file():
    if not check_login_required():
        return
    file_path = filedialog.askopenfilename()
    if not file_path:
        return
    if not has_permission(file_path, logged_in_user, "read"):
        messagebox.showerror("Access Denied", "You do not have read permission for this file.")
        return
    content = decrypt_file(logged_in_user, file_path)
    messagebox.showinfo("File Content", content)

def write_file():
    if not check_login_required():
        return
    popup = tk.Toplevel(root)
    popup.title("Write File")

    tk.Label(popup, text="Enter text to save:").pack(pady=5)
    text_area = tk.Text(popup, height=10, width=50)
    text_area.pack(padx=10, pady=10)

    def save_file():
        content = text_area.get("1.0", "end-1c")
        popup.destroy()
        if detect_malware(content):
            messagebox.showerror("Error", "Malicious content detected! File not saved.")
            return
        if not validate_input(content):
            messagebox.showerror("Error", "Suspicious content detected! Chance of Buffer Overflow. File not saved.")
            return
        file_path = filedialog.asksaveasfilename(defaultextension=".enc")
        if file_path:
            encrypt_file(logged_in_user, file_path, content)
            set_initial_permissions(file_path, logged_in_user)  # Set creator as initial reader/writer
            messagebox.showinfo("Success", "File saved and encrypted.")
```

```
    tk.Button(popup, text="Save", command=save_file).pack(pady=10)

def view_metadata():
    if not check_login_required():
        return
    file_path = filedialog.askopenfilename()
    if not file_path:
        return
    if not has_permission(file_path, logged_in_user, "read"):
        messagebox.showerror("Access Denied", "You do not have read permission for this file.")
        return
    metadata = get_file_metadata(file_path)
    messagebox.showinfo("File Metadata", f"Name: {metadata['name']}\nSize: {metadata['size']} bytes\nCreator:
{metadata['creator']}\nCreated: {metadata['created']}\nModified: {metadata['modified']}")

def scan_file():
    if not check_login_required():
        return
    file_path = filedialog.askopenfilename()
    if not file_path:
        return
    if not has_permission(file_path, logged_in_user, "read"):
        messagebox.showerror("Access Denied", "You do not have read permission for this file.")
        return
    is_malware = detect_malware(decrypt_file(logged_in_user, file_path))
    if is_malware:
        messagebox.showerror("Threat Detected", f"Malware detected in file: {file_path}")
        log_threat(file_path, "Malware detected")
    else:
        messagebox.showinfo("Safe", "No threats detected.")

def manage_permissions():
    if not check_login_required():
        return
    file_path = filedialog.askopenfilename()
    if not file_path:
        return
    current_perms = get_permissions(file_path)
    # Fetch the first user with write permission (or read if no writers)
    owner = current_perms["write"][0] if current_perms["write"] else (current_perms["read"][0] if current_perms["read"] else
None)
    if not owner or owner != logged_in_user:
        messagebox.showerror("Access Denied", "Only the first authorized user can manage permissions.")
        return

    perm_window = tk.Toplevel(root)
    perm_window.title(f"Manage Permissions for {os.path.basename(file_path)}")

    user_list = list(users.keys())
```

```
    current_perms = get_permissions(file_path)

    read_listbox = tk.Listbox(perm_window, height=10, width=20)
    write_listbox = tk.Listbox(perm_window, height=10, width=20)
    read_listbox.grid(row=0, column=0, padx=10, pady=10)
    write_listbox.grid(row=0, column=1, padx=10, pady=10)

    tk.Label(perm_window,  text="Readers").grid(row=1,  column=0)
    tk.Label(perm_window, text="Writers").grid(row=1, column=1)

    def update_listboxes():
        read_listbox.delete(0, tk.END)
        write_listbox.delete(0, tk.END)
        for user in current_perms["read"]:
            read_listbox.insert(tk.END, user)
        for user in current_perms["write"]:
            write_listbox.insert(tk.END, user)

    update_listboxes()

    def add_user(permission_type):
        username = simpledialog.askstring(f"Add {permission_type.capitalize()}", f"Enter username to add to {permission_type}:")
        if not username or username not in user_list:
            messagebox.showerror("Error", "Invalid username.")
            return
        if add_permission(file_path, username, permission_type):
            nonlocal current_perms
            current_perms = get_permissions(file_path)
            update_listboxes()

    def remove_user(permission_type):
        username = simpledialog.askstring(f"Remove {permission_type.capitalize()}", f"Enter username to remove from
{permission_type}:")
        if not username or username not in user_list:
            messagebox.showerror("Error", "Invalid username.")
            return
        if remove_permission(file_path, username, permission_type):
            nonlocal current_perms
            current_perms = get_permissions(file_path)
            update_listboxes()

    tk.Button(perm_window, text="Add Reader", command=lambda: add_user("read")).grid(row=2, column=0, pady=5)
    tk.Button(perm_window, text="Add Writer", command=lambda: add_user("write")).grid(row=2, column=1, pady=5)
    tk.Button(perm_window, text="Remove Reader", command=lambda: remove_user("read")).grid(row=3, column=0, pady=5)
    tk.Button(perm_window, text="Remove Writer", command=lambda: remove_user("write")).grid(row=3, column=1, pady=5)

# GUI setup
root = tk.Tk()
root.title("Secure File Management System")
```

```
# Tabs setup
notebook = tk.ttk.Notebook(root)

# Authentication tab
auth_frame = tk.Frame(notebook)
notebook.add(auth_frame, text="Authentication")

tk.Label(auth_frame, text="Register").pack()
reg_username_entry = tk.Entry(auth_frame)
reg_username_entry.pack()
reg_password_entry = tk.Entry(auth_frame, show="*")
reg_password_entry.pack()
tk.Button(auth_frame, text="Register", command=register).pack()

tk.Label(auth_frame, text="Login").pack()
login_username_entry = tk.Entry(auth_frame)
login_username_entry.pack()
login_password_entry = tk.Entry(auth_frame, show="*")
login_password_entry.pack()
otp_entry = tk.Entry(auth_frame)
otp_entry.pack()
tk.Button(auth_frame, text="Send OTP", command=send_otp).pack()
tk.Button(auth_frame, text="Login", command=login).pack()

# File operations tab
file_ops_frame = tk.Frame(notebook)
notebook.add(file_ops_frame, text="File Operations")

button_frame = tk.Frame(file_ops_frame)
button_frame.pack(expand=True)

tk.Button(button_frame, text="Read File", command=read_file).grid(row=0, column=0, padx=10, pady=10)
tk.Button(button_frame, text="Write File", command=write_file).grid(row=0, column=1, padx=10, pady=10)
tk.Button(button_frame, text="View Metadata", command=view_metadata).grid(row=1, column=0, padx=10, pady=10)
tk.Button(button_frame, text="Scan File", command=scan_file).grid(row=1, column=1, padx=10, pady=10)
tk.Button(button_frame, text="Manage Permissions", command=manage_permissions).grid(row=2, column=0, columnspan=2,
padx=10, pady=10)

notebook.pack(expand=True, fill="both")
root.mainloop()
```

module1_auth.py:

```
import   bcrypt
import  hashlib
import smtplib
import  base64
import json
from tkinter import messagebox
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
import random
import string
import time

# Store static OTPs and their expiration times
static_otps = {}

# File to store users persistently
USERS_FILE = "users.json"

# Helper function to hash passwords
def hash_password(password):
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt(10))

# Load users from file
def load_users():
    global users
    try:
        with open(USERS_FILE, 'r') as f:
            loaded_users = json.load(f)
            # Convert hex/base64 strings back to bytes where necessary
            users = {}
            for username, data in loaded_users.items():
                users[username] = {
                    "password": bytes.fromhex(data["password"]), # Stored as hex
                    "secret_key": data["secret_key"]  # Already a base64 string
                }
    except FileNotFoundError:
        # Default user if file doesn't exist
        password_hash = hash_password("123")
        users = {
            "user": {
                "password": password_hash,
                "secret_key": "nmgWMPkXMnRFGY6sHqShTFeiCSg5x7VMCKjVmcDFLIk=",
            },
        }
        save_users()  # Save initial state

# Save users to file
def save_users():
    # Convert bytes to hex/base64 for JSON serialization
```

```
    serializable_users = {}
    for username, data in users.items():
        serializable_users[username] = {
            "password": data["password"].hex(),  # Convert bytes to hex
            "secret_key": data["secret_key"]  # Already a base64 string
        }
    with open(USERS_FILE, 'w') as f:
        json.dump(serializable_users, f)

# Load users at module initialization
users = {}
load_users()

# Helper function to verify hashed passwords
def verify_password(password, hashed):
    return bcrypt.checkpw(password.encode('utf-8'), hashed)

# Function to register a new user
def register_user(username, password):
    if username in users:
        return "User already exists."

    # Hash the password
    hashed_password = hash_password(password)

    # Generate a unique secret key based on the username
    hashed_username = hashlib.sha256(username.encode('utf-8')).digest()
    secret_key = base64.urlsafe_b64encode(hashed_username[:32])

    # Store the hashed password and valid Fernet key in the users dictionary
    users[username] = {
        "password": hashed_password,
        "secret_key":  secret_key.decode('utf-8'),
    }
    save_users()  # Persist changes
    print(f"Generated secret key for {username}: {secret_key.decode('utf-8')}")
    return "User registered successfully."

# Function to generate a static OTP
# We are generating static random otp just for demonstration, otherwise we know that python libraries for time based otp
# such as pyotp exists which are more secure. We can easily swap out otp generation using such advanced libraries easily as our
code is modular
def generate_static_otp(username, validity=300):
    """Generate a static OTP valid for a specific duration (default: 300 seconds)."""
    if username not in users:
        return "User not found."

    otp = ''.join(random.choices(string.digits, k=6))  # Generate a 6-digit OTP
    expiration_time = time.time() + validity
```

```
    static_otps[username] = {"otp": otp, "expires_at": expiration_time}
    user_email = "user-email@example.com"  # Replace with actual email logic if needed
    try:
        # SMTP Configuration
        sender_email = "sender-email@example.com"
        sender_password = ""  # Replace with your 16-digit app password
        smtp_server = "smtp.gmail.com"
        smtp_port = 587

        # Email Message
        subject = "Your Static OTP Code"
        message = f"Hello {username},\n\nYour static OTP code is: {otp}\n\nThis code will expire in {validity//60}
minutes.\n\nThank you!"

        msg = MIMEMultipart()
        msg["From"] = sender_email
        msg["To"] = user_email
        msg["Subject"] = subject
        msg.attach(MIMEText(message,  "plain"))

        # Sending Email
        server = smtplib.SMTP(smtp_server, smtp_port)
        server.starttls()
        server.login(sender_email, sender_password)
        server.sendmail(sender_email, user_email, msg.as_string())
        server.quit()

        messagebox.showinfo("OTP Sent", f"Static OTP has been sent to {user_email}")
    except Exception as e:
        with open("threat_log.txt", "a") as log_file:
            log_file.write(f"Failed to send static OTP: {e}\n")
        messagebox.showerror("Error", f"Failed to send static OTP: {e}")

    return otp

# Function to validate a static OTP
def validate_static_otp(username, otp):
    if username not in static_otps:
        return False
    otp_data = static_otps[username]
    if time.time() > otp_data["expires_at"]:
        del static_otps[username]  # Remove expired OTP
        return False
    if otp_data["otp"] == otp:
        del static_otps[username]  # OTP is valid, remove it after use
        return True
    return False

# Updated authentication to support static OTPs
```

```
def authenticate_user(username, password, otp):
    if username not in users:
        return False
    user = users[username]
    if not verify_password(password, user["password"]):
        return False
    # Check static OTP
    if validate_static_otp(username, otp):
        return True
    return False
```

module2_p1_file_ops.py:

```
from cryptography.fernet import Fernet
import os
import json
from datetime import datetime
from module1_auth import users

# Store file-specific symmetric keys
file_symmetric_keys = {}

def load_file_keys():
    global file_symmetric_keys
    try:
        with open('file_keys.json', 'r') as f:
            file_symmetric_keys = json.load(f)
    except FileNotFoundError:
        file_symmetric_keys = {}

def save_file_keys():
    with open('file_keys.json', 'w') as f:
        json.dump(file_symmetric_keys, f)

load_file_keys()

def get_cipher_suite(username):
    user_data = users.get(username)
    if not user_data or "secret_key" not in user_data:
        raise ValueError(f"No valid secret_key found for user: {username}")
    encryption_key = user_data["secret_key"].encode("utf-8")
    return Fernet(encryption_key)

def encrypt_file(username, file_path, content):
    cipher_suite = get_cipher_suite(username)
    # Generate a new symmetric key for this file
```

```
    file_symmetric_key = Fernet.generate_key()
    fernet_file = Fernet(file_symmetric_key)
    encrypted_content = fernet_file.encrypt(content.encode('utf-8'))
    with open(file_path, 'wb') as f:
        f.write(encrypted_content)
    # Encrypt the file's symmetric key with the owner's secret key
    encrypted_file_key = cipher_suite.encrypt(file_symmetric_key)
    # Store it as hex string for JSON compatibility
    file_symmetric_keys[file_path] = {
        "key":  encrypted_file_key.hex(),
        "creator": username
    }
    save_file_keys()

def decrypt_file(username, file_path):
    try:
        file_data = file_symmetric_keys[file_path]
        encrypted_file_key_hex = file_data["key"]
        encrypted_file_key = bytes.fromhex(encrypted_file_key_hex)
    except KeyError:
        return f"Error: File not found or no key available."
    cipher_suite = get_cipher_suite(username)
    try:
        file_symmetric_key = cipher_suite.decrypt(encrypted_file_key)
    except Exception as e:
        return f"Error decrypting file key: {e}"
    fernet_file  =  Fernet(file_symmetric_key)
    with open(file_path, 'rb') as f:
        encrypted_content = f.read()
    return  fernet_file.decrypt(encrypted_content).decode('utf-8')

def get_file_metadata(file_path):
    metadata = os.stat(file_path)
    creator = file_symmetric_keys.get(file_path, {}).get("creator", os.getlogin())
    return {
        "name": os.path.basename(file_path),
        "creator": creator,
        "size": metadata.st_size,
        "created": datetime.fromtimestamp(metadata.st_ctime).strftime('%d/%m/%Y'),
        "modified": datetime.fromtimestamp(metadata.st_mtime).strftime('%d/%m/%Y'),
    }
```

module2_p2_access_matrix.py:

```
import json
from module2_p1_file_ops import file_symmetric_keys

permissions = {}

def load_permissions():
    global permissions
    try:
        with open('access_matrix.json', 'r') as f:
            permissions = json.load(f)
    except FileNotFoundError:
        permissions = {}

def save_permissions():
    with open('access_matrix.json', 'w') as f:
        json.dump(permissions, f)

load_permissions()

def get_permissions(file_path):
    return permissions.get(file_path, {"read": [], "write": []})

def set_initial_permissions(file_path, creator):
    permissions[file_path] = {
        "read": [creator],
        "write": [creator]
    }
    save_permissions()

def add_permission(file_path, username, permission_type):
    if file_path not in permissions:
        return False
    creator = file_symmetric_keys.get(file_path, {}).get("creator")
    if not creator:
        return False
    if permission_type not in ["read", "write"]:
        return False
    if username not in permissions[file_path][permission_type]:
        permissions[file_path][permission_type].append(username)
        save_permissions()
    return True

def remove_permission(file_path, username, permission_type):
    if file_path not in permissions:
        return False
    creator = file_symmetric_keys.get(file_path, {}).get("creator")
    if not creator or username == creator:
        return False  # Cannot remove creator's permissions
    if permission_type not in ["read", "write"]:
```

```
        return False
    if username in permissions[file_path][permission_type]:
        permissions[file_path][permission_type].remove(username)
        save_permissions()
    return True

def has_permission(file_path, username, permission_type):
    if file_path not in permissions:
        return False
    return username in permissions[file_path].get(permission_type, [])
```

module3_threat_detect.py:

```
import hashlib
import re

# Mock antivirus hash database (to simulate malware detection)
KNOWN_MALICIOUS_HASHES = {
    "098f6bcd4621d373cade4e832627b4f6",  # Example hash for "test"
}

def calculate_file_hash(file_path):
    hash_md5 = hashlib.md5()
    try:
        with open(file_path, "rb") as f:
            for chunk in iter(lambda: f.read(4096), b""):
                hash_md5.update(chunk)
    except Exception as e:
        return None, str(e)
    return hash_md5.hexdigest(), None

def detect_malware(file_content):
    file_hash = hashlib.md5(file_content.encode('utf-8')).hexdigest()
    if file_hash in KNOWN_MALICIOUS_HASHES:
        with open("threat_log.txt", "a") as log_file:
            log_file.write(f"Malware Detected: File hash {file_hash} matches known malicious hash.\n")
        return True
    return False

def validate_input(input_data):
    """
    Validate input against common buffer overflow patterns.
    """
    patterns = [
        r"(?:\x90{3,})",  # NOP sleds
        r"(?:A{5,})",     # Long sequence of 'A's

        # ** Other such common patterns **
        # r"(?:[\x00-\x1F]{5,})",  # Repeated control characters: Often used in exploits
```

```
        # r"(?:\xCC{3,})",          # INT3 breakpoints: Debug interrupt instructions
        # r"(?:%(?:[0-9A-Fa-f]{2}){5,})",  # Repeated hex-encoded bytes (e.g., %41%41%41): URL-encoded overflow
        # r"(?:\xEB.{1,10}\xEB)",  # Short JMP instructions: Used to redirect execution
        # r"(?:[ -~]{100,})"       # Excessive printable ASCII: Unnaturally long strings
        # **                        **
    ]
    for pattern in patterns:
        if re.search(pattern, input_data):
            with open("threat_log.txt", "a") as log_file:
                log_file.write(f"Suspicious Pattern Detected: {pattern} found in input.\n")
            return False
    return True

def log_threat(file_path, threat_type):
    with open("threat_log.txt", "a") as log_file:
        log_file.write(f"Threat: {threat_type}\nFile: {file_path}\n\n")
```